Моделирование сетей передачи данных

Лабораторная работа №3

Авдадаев Джамал Геланиевич

10 октября 2026

Российский университет дружбы народов, Москва, Россия

Цели и задачи работы

Цель лабораторной работы

Освоить Python API Mininet и организовать воспроизводимый эксперимент по измерению пропускной способности виртуальной сети с помощью iPerf3.

Задачи лабораторной работы

- Создать и проверить топологию h1 — s3 — h2
- Настроить ограничения ресурсов и канала
- Сравнить ICMP-показатели двух топологий
- Запустить iPerf3 и сохранить результаты в JSON
- Построить графики и автоматизировать повторный запуск

Ход выполнения работы

Подготовка топологии Mininet

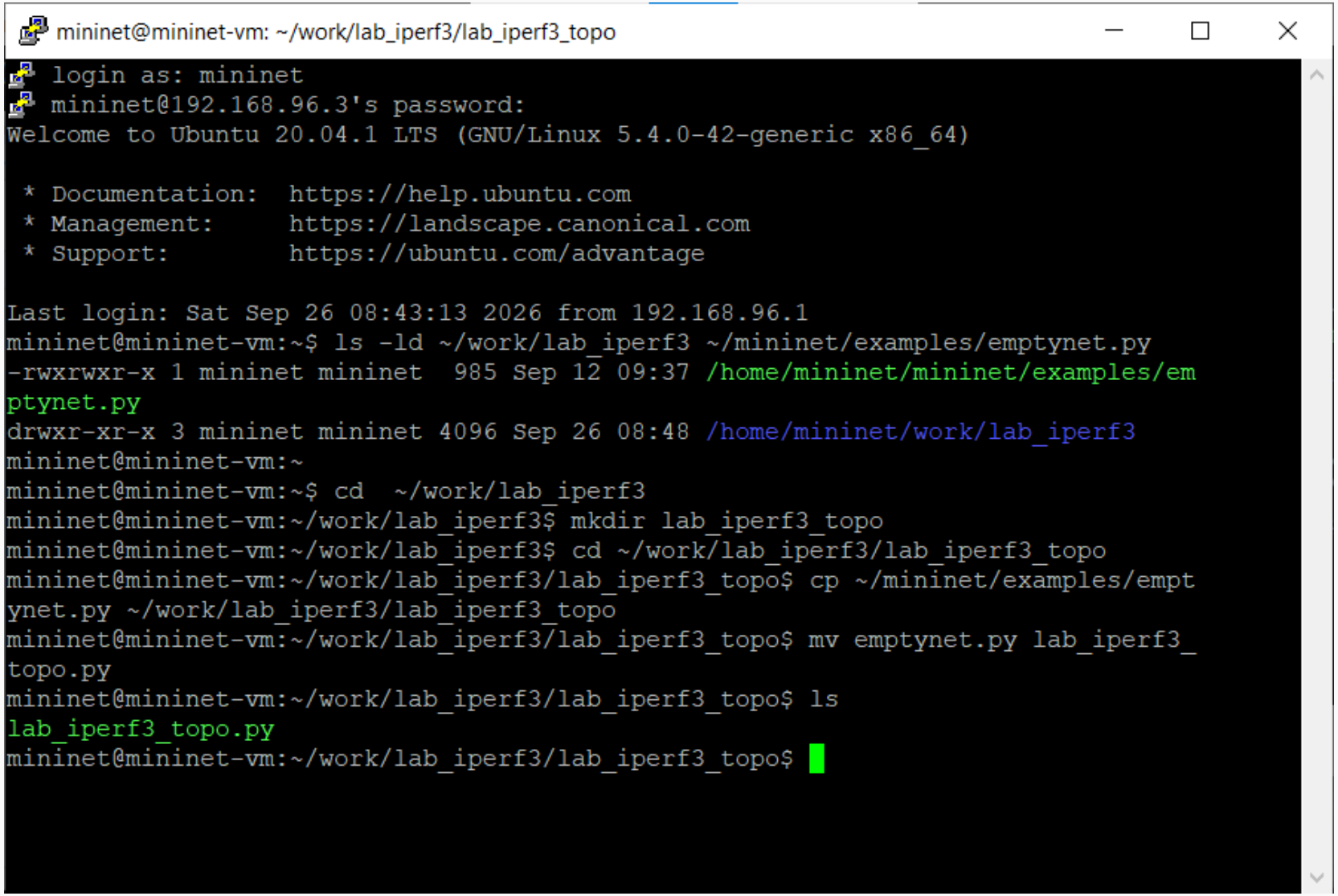


Рис. 1: Подготовка рабочей папки и копирование emptynet.py

Создание топологии с помощью Python

mininet@mininet-vm: ~/work/lab_iperf3/lab_iperf3_topo

```
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ cat lab_iperf3_topo.py
c≡cat: command not found
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ cat lab_iperf3_topo.py
#!/usr/bin/env python

"""
This example shows how to create an empty Mininet object
(without a topology object) and add nodes to it manually.
"""

from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info

def emptyNet():

    "Create an empty network and add nodes to it."

    net = Mininet( controller=Controller, waitConnected=True )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1' )
    h2 = net.addHost( 'h2', ip='10.0.0.2' )

    info( '*** Adding switch\n' )
    s3 = net.addSwitch( 's3' )

    info( '*** Creating links\n' )
    net.addLink( h1, s3 )
    net.addLink( h2, s3 )

    info( '*** Starting network\n')
    net.start()

    info( '*** Running CLI\n' )
    CLI( net )

    info( '*** Stopping network' )
    net.stop()


if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$
```

Рис. 2: Исходный скрипт создания сети

Запуск исходной топологии

```
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ sudo python3 lab_iperf3_topo.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s3 ...
*** Waiting for switches to connect
s3
*** Running CLI
*** Starting CLI:
mininet>
```

Рис. 3: Запуск исходной топологии через Python API

Проверка связности хостов

```
mininet> net
h1 h1-eth0:s3-eth1
h2 h2-eth0:s3-eth2
s3 lo:  s3-eth1:h1-eth0 s3-eth2:h2-eth0
c0
mininet> links
h1-eth0<->s3-eth1 (OK OK)
h2-eth0<->s3-eth2 (OK OK)
mininet> dump
<Host h1: h1-eth0:10.0.0.1 pid=936>
<Host h2: h2-eth0:10.0.0.2 pid=939>
<OVSSwitch s3: lo:127.0.0.1,s3-eth1:None,s3-eth2:None pid=944>
<Controller c0: 127.0.0.1:6653 pid=929>
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet>
```

Рис. 4: Проверка связности и состава сети

Получение адресных параметров h1

mininet@mininet-vm: ~/work/lab_iperf3/lab_iperf3_topo

GNU nano 4.8 lab_iperf3_topo.py Modified

```
    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1' )
    h2 = net.addHost( 'h2', ip='10.0.0.2' )

    info( '*** Adding switch\n' )
    s3 = net.addSwitch( 's3' )

    info( '*** Creating links\n' )
    net.addLink( h1, s3 )
    net.addLink( h2, s3 )

    info( '*** Starting network\n')
    net.start()

    print("Host", h1.name, "has IP address", h1.IP(), "and MAC address", h1.MAC())

    info( '*** Running CLI\n' )
    CLI( net )

    info( '*** Stopping network' )
    net.stop()
```

^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell ^_ Go To Line

Рис. 5: Добавление вывода параметров хоста h1

Вывод IP- и MAC-адреса хоста h1

```
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ sudo python3 lab_iperf3_topo.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s3 ...
*** Waiting for switches to connect
s3
Host h1 has IP address 10.0.0.1 and MAC address fe:f4:57:84:87:e7
*** Running CLI
*** Starting CLI:
mininet>
```

Рис. 6: Вывод IP- и MAC-адреса h1

Добавление вывода адресов h2

mininet@mininet-vm: ~/work/lab_iperf3/lab_iperf3_topo

GNU nano 4.8 lab_iperf3_topo.py Modified

```
    info( '*** Adding switch\n' )
    s3 = net.addSwitch( 's3' )

    info( '*** Creating links\n' )
    net.addLink( h1, s3 )
    net.addLink( h2, s3 )

    info( '*** Starting network\n')
    net.start()

    print("Host", h1.name, "has IP address", h1.IP(), "and MAC address", h1.MAC())
    print("Host", h2.name, "has IP address", h2.IP(), "and MAC address", h2.MAC())

    info( '*** Running CLI\n' )
    CLI( net )

    info( '*** Stopping network' )
    net.stop()


if __name__ == '__main__':
    setLogLevel( 'info' )
    emptyNet()
```

^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell ^_ Go To Line

Рис. 7: Добавление вывода параметров второго хоста

Получение адресов обоих хостов

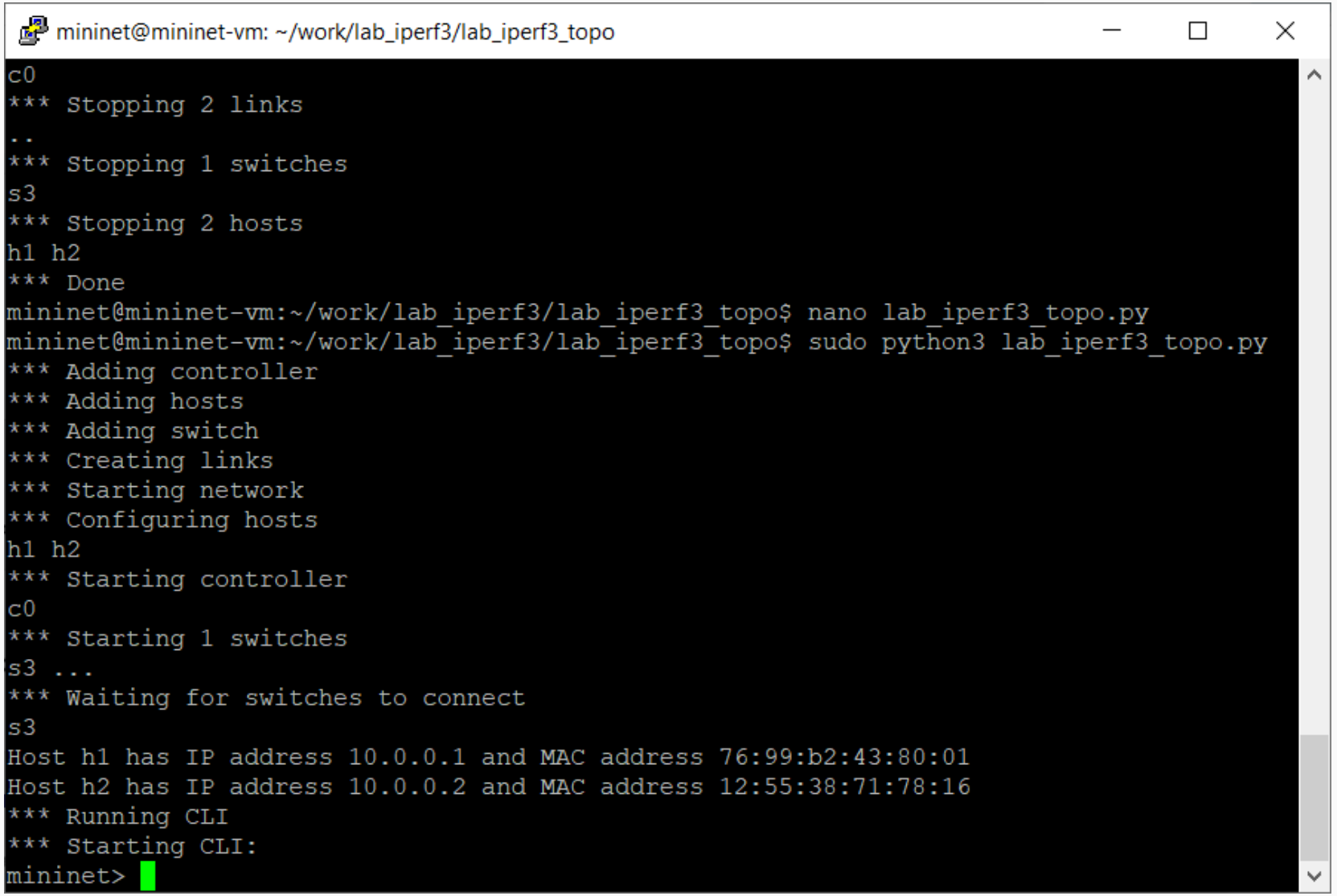


Рис. 8: Результаты получения адресов обоих хостов

Моделирование ограничений

Импорт CPULimitedHost и TCLink

mininet@mininet-vm: ~/work/lab_iperf3/lab_iperf3_topo

```
  GNU nano 4.8                        lab_iperf3_topo2.py
#!/usr/bin/env python

"""
This example shows how to create an empty Mininet object
(without a topology object) and add nodes to it manually.
"""

from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info

from mininet.node import CPULimitedHost
from mininet.link import TCLink
def emptyNet():

    "Create an empty network and add nodes to it."

    net = Mininet( controller=Controller, waitConnected=True )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
                                [ Wrote 50 lines ]
^G Get Help   ^O Write Out  ^W Where Is   ^K Cut Text   ^J Justify    ^C Cur Pos
^X Exit       ^R Read File  ^\ Replace    ^U Paste Text ^T To Spell   ^_ Go To Line
```

Рис. 9: Подключение классов CPULimitedHost и TCLink

Настройка ограничиваемых узлов и линий

mininet@mininet-vm: ~/work/lab_iperf3/lab_iperf3_topo

```
  GNU nano 4.8                    lab_iperf3_topo2.py                 Modified
#!/usr/bin/env python

"""
This example shows how to create an empty Mininet object
(without a topology object) and add nodes to it manually.
"""

from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info

from mininet.node import CPULimitedHost
from mininet.link import TCLink
def emptyNet():

    "Create an empty network and add nodes to it."

    net = Mininet(
    controller=Controller,
    waitConnected=True,
    host=CPULimitedHost,
    link=TCLink
    )

^G Get Help  ^O Write Out  ^W Where Is  ^K Cut Text  ^J Justify  ^C Cur Pos
^X Exit      ^R Read File  ^\ Replace   ^U Paste Text ^T To Spell ^_ Go To Line
```

Рис. 10: Настройка классов узлов и соединений Mininet

Ограничение процессорных ресурсов

mininet@mininet-vm: ~/work/lab_iperf3/lab_iperf3_topo

```
 GNU nano 4.8                    lab_iperf3_topo2.py                 Modified
from mininet.cli import CLI
from mininet.log import setLogLevel, info

from mininet.node import CPULimitedHost
from mininet.link import TCLink
def emptyNet():

    "Create an empty network and add nodes to it."

    net = Mininet(
    controller=Controller,
    waitConnected=True,
    host=CPULimitedHost,
    link=TCLink
    )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1', cpu=0.5 )
    h2 = net.addHost( 'h2', ip='10.0.0.2', cpu=0.45 )

    info( '*** Adding switch\n' )

^G Get Help   ^O Write Out  ^W Where Is   ^K Cut Text   ^J Justify    ^C Cur Pos
^X Exit       ^R Read File  ^\ Replace    ^U Paste Text ^T To Spell   ^_ Go To Line
```

Рис. 11: Назначение ограничений CPU хостам

Настройка характеристик канала

mininet@mininet-vm: ~/work/lab_iperf3/lab_iperf3_topo

GNU nano 4.8 lab_iperf3_topo2.py Modified

```
def emptyNet():

    "Create an empty network and add nodes to it."

    net = Mininet(
    controller=Controller,
    waitConnected=True,
    host=CPULimitedHost,
    link=TCLink
    )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1', cpu=0.5 )
    h2 = net.addHost( 'h2', ip='10.0.0.2', cpu=0.45 )

    info( '*** Adding switch\n' )
    s3 = net.addSwitch( 's3' )

    info( '*** Creating links\n' )
    net.addLink(h1, s3, bw=10, delay='5ms', max_queue_size=1000, loss=10, use_htb=True)
    net.addLink( h2, s3 )
```

^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell ^_ Go To Line

Рис. 12: Настройка скорости, задержки и потерь канала

Запуск сети с ограничениями

```
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ sudo python3 lab_iperf3_topo2.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
(10.00Mbit 5ms delay 10.00000% loss) (10.00Mbit 5ms delay 10.00000% loss) *** Starting
network
*** Configuring hosts
h1 (cfs 200000/100000us) h2 (cfs 180000/100000us)
*** Starting controller
c0
*** Starting 1 switches
s3 (10.00Mbit 5ms delay 10.00000% loss) ...(10.00Mbit 5ms delay 10.00000% loss)
*** Waiting for switches to connect
s3
Host h1 has IP address 10.0.0.1 and MAC address 36:b2:2d:5f:1f:db
Host h2 has IP address 10.0.0.2 and MAC address 72:aa:42:bb:87:62
*** Running CLI
*** Starting CLI:
mininet>
```

Рис. 13: Запуск сети с заданными ограничениями

Проверка параметров с помощью tc

```
mininet> links
h1-eth0<->s3-eth1 (OK OK)
h2-eth0<->s3-eth2 (OK OK)
mininet> h1 tc qdisc show dev h1-eth0
qdisc htb 5: root refcnt 2 r2q 10 default 0x1 direct_packets_stat 0 direct_qlen 1000
qdisc netem 10: parent 5:1 limit 1000 delay 5.0ms loss 10%
mininet> h1 tc class show dev h1-eth0
class htb 5:1 root leaf 10: prio 0 rate 10Mbit ceil 10Mbit burst 15Kb cburst 1600b
mininet>
```

Рис. 14: Проверка HTB и netem через tc

Проверка работоспособности

ICMP-тест сети с ограничениями

```
mininet> h1 ping -c 20 -i 0.2 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=628 ms
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1035 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=167 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=831 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=370 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=11.1 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=15.8 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=20.0 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=23.1 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=11.0 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=10.5 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=10.5 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=10.5 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=11.0 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=11.5 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=10.5 ms
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=11.1 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 17 received, 15% packet loss, time 4003ms
rtt min/avg/max/mdev = 10.454/187.525/1034.513/318.533 ms, pipe 5
mininet>
```

Рис. 15: ICMP-измерения в ограниченной сети

ICMP-тест исходной сети

```
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ sudo python3 lab_iperf3_topo.py
*** Adding controller
*** Adding hosts
*** Adding switch
*** Creating links
*** Starting network
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s3 ...
*** Waiting for switches to connect
s3
Host h1 has IP address 10.0.0.1 and MAC address 46:b5:5c:ba:c5:38
Host h2 has IP address 10.0.0.2 and MAC address ca:50:b0:76:cb:2a
*** Running CLI
*** Starting CLI:
mininet> h1 ping -c 20 -i 0.2 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.52 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.559 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.071 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.061 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.081 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.071 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.072 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.058 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.081 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.074 ms
64 bytes from 10.0.0.2: icmp_seq=11 ttl=64 time=0.070 ms
64 bytes from 10.0.0.2: icmp_seq=12 ttl=64 time=0.080 ms
64 bytes from 10.0.0.2: icmp_seq=13 ttl=64 time=0.072 ms
64 bytes from 10.0.0.2: icmp_seq=14 ttl=64 time=0.098 ms
64 bytes from 10.0.0.2: icmp_seq=15 ttl=64 time=0.118 ms
64 bytes from 10.0.0.2: icmp_seq=16 ttl=64 time=0.129 ms
64 bytes from 10.0.0.2: icmp_seq=17 ttl=64 time=0.056 ms
64 bytes from 10.0.0.2: icmp_seq=18 ttl=64 time=0.076 ms
64 bytes from 10.0.0.2: icmp_seq=19 ttl=64 time=0.111 ms
64 bytes from 10.0.0.2: icmp_seq=20 ttl=64 time=0.065 ms

--- 10.0.0.2 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 4381ms
rtt min/avg/max/mdev = 0.056/0.176/1.518/0.325 ms
mininet>
```

Рис. 16: ICMP-измерения в сети без ограничений

Воспроизводимый эксперимент

Подготовка автоматизированного эксперимента

```
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ cp lab_iperf3_topo2.py lab_iperf3
.py
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ mkdir -p ~/work/lab_iperf3/iperf3
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ mv lab_iperf3.py  ~/work/lab_iper
f3/iperf3/
mininet@mininet-vm:~/work/lab_iperf3/lab_iperf3_topo$ cd ~/work/lab_iperf3/iperf3
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ pwd
/home/mininet/work/lab_iperf3/iperf3
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ ls -lh
total 4.0K
-rwxrwxr-x 1 mininet mininet 1.4K Oct 10 07:28 lab_iperf3.py
mininet@mininet-vm:~/work/lab_iperf3/iperf3$
```

Рис. 17: Создание каталога автоматизированного эксперимента

Настройка Python-скрипта iPerf3

mininet@mininet-vm: ~/work/lab_iperf3/iperf3

```
 GNU nano 4.8                                lab_iperf3.py
import time
from mininet.net import Mininet
from mininet.node import Controller
from mininet.cli import CLI
from mininet.log import setLogLevel, info

from mininet.link import TCLink
def emptyNet():

    "Create an empty network and add nodes to it."

    net = Mininet(
    controller=Controller,
    waitConnected=True,
    link=TCLink
    )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1')
    h2 = net.addHost( 'h2', ip='10.0.0.2')


^G Get Help   ^O Write Out  ^W Where Is   ^K Cut Text   ^J Justify    ^C
^X Exit       ^R Read File  ^\ Replace    ^U Paste Text ^T To Spell   ^
```

Рис. 18: Подготовка скрипта без ограничений CPU

Конфигурация каналов по 100 Мбит/с

mininet@mininet-vm: ~/work/lab_iperf3/iperf3

GNU nano 4.8 lab_iperf3.py

```
    net = Mininet(
    controller=Controller,
    waitConnected=True,
    link=TCLink
    )

    info( '*** Adding controller\n' )
    net.addController( 'c0' )

    info( '*** Adding hosts\n' )
    h1 = net.addHost( 'h1', ip='10.0.0.1')
    h2 = net.addHost( 'h2', ip='10.0.0.2')

    info( '*** Adding switch\n' )
    s3 = net.addSwitch( 's3' )

    info( '*** Creating links\n' )
    net.addLink(h1, s3, bw=100, delay='75ms', loss=0, use_htb=False)
    net.addLink(h2, s3, bw=100, delay='75ms', loss=0, use_htb=False)

    info( '*** Starting network\n')
    net.start()

    print("Host", h1.name, "has IP address", h1.IP(), "and MAC address",
```

^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell ^_

Рис. 19: Настройка обоих каналов на 100 Мбит/с

Автоматический запуск сервера и клиента

mininet@mininet-vm: ~/work/lab_iperf3/iperf3

GNU nano 4.8 lab_iperf3.py Modified

```
    info( '*** Adding switch\n' )
    s3 = net.addSwitch( 's3' )

    info( '*** Creating links\n' )
    net.addLink(h1, s3, bw=100, delay='75ms', loss=0, use_htb=False)
    net.addLink(h2, s3, bw=100, delay='75ms', loss=0, use_htb=False)

    info( '*** Starting network\n')
    net.start()

    print("Host", h1.name, "has IP address", h1.IP(), "and MAC address", h1.MAC())
    print("Host", h2.name, "has IP address", h2.IP(), "and MAC address", h2.MAC())

    info('*** Traffic generation\n')
    h2.cmdPrint('iperf3 -s -D -1')
    time.sleep(10)
    h1.cmdPrint('iperf3 -c', h2.IP(), '-J > iperf_result.json')

    # info('*** Running CLI\n')
    # CLI(net)

    info( '*** Stopping network' )
    net.stop()
```

^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell ^_ Go To Line

Рис. 20: Программный запуск iPerf3 и сохранение JSON

Выполнение TCP-эксперимента

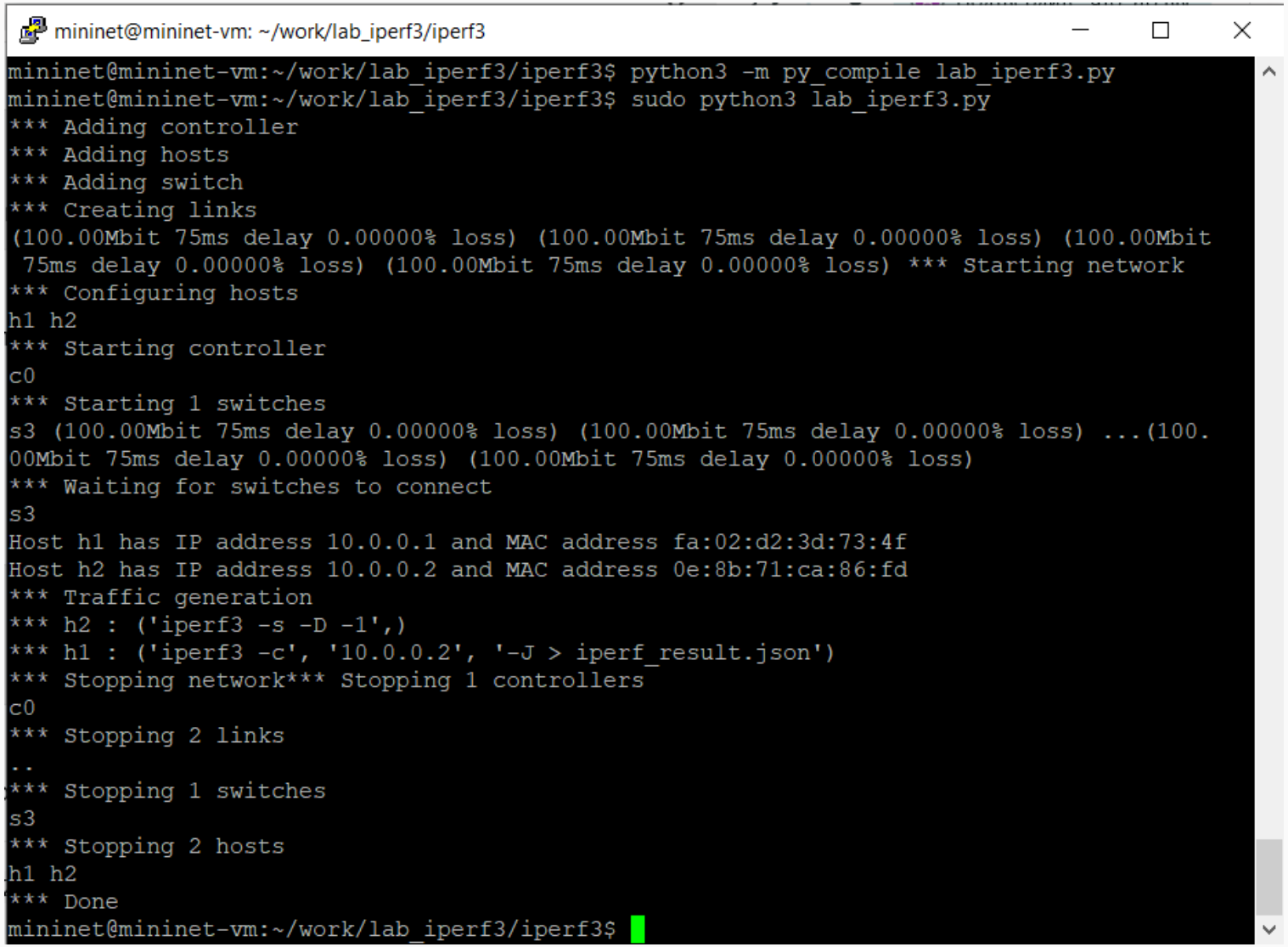


Рис. 21: Выполнение автоматизированного эксперимента

Проверка результатов в JSON

```
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ ls -lh iperf_result.json
-rw-r--r-- 1 root root 7.6K Oct 10 07:48 iperf_result.json
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ python3 -m json.tool iperf_result.json  > /de
v/null && echo "JSON VALID"
JSON VALID
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ python3 -c "import json; d=json.load(open('ip
erf_result.json')); print('Error:',d.get('error','none')); print('Bits per second:',d.get(
'end',{}).get('sum_received',{}).get('bits_per_second','missing'))"
Error: none
Bits per second: 9967259.812057052
mininet@mininet-vm:~/work/lab_iperf3/iperf3$
```

Рис. 22: Проверка первого JSON-результата

Построение графиков iPerf3

```
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ sudo chown mininet:mininet iperf_result.json
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ plot_iperf.sh iperf_result.json
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ ls -lh results/
total 88K
-rw-rw-r-- 1 mininet mininet  528 Oct 10 07:54 1.dat
-rw-rw-r-- 1 mininet mininet 9.5K Oct 10 07:54 bytes.pdf
-rw-rw-r-- 1 mininet mininet 9.5K Oct 10 07:54 cwnd.pdf
-rw-rw-r-- 1 mininet mininet 8.9K Oct 10 07:54 MTU.pdf
-rw-rw-r-- 1 mininet mininet 8.8K Oct 10 07:54 retransmits.pdf
-rw-rw-r-- 1 mininet mininet 8.8K Oct 10 07:54 RTT.pdf
-rw-rw-r-- 1 mininet mininet 9.1K Oct 10 07:54 RTT_Var.pdf
-rw-rw-r-- 1 mininet mininet 9.3K Oct 10 07:54 throughput.pdf
mininet@mininet-vm:~/work/lab_iperf3/iperf3$
```

Рис. 23: Генерация PDF-графиков

Визуализация результатов

Изменение пропускной способности

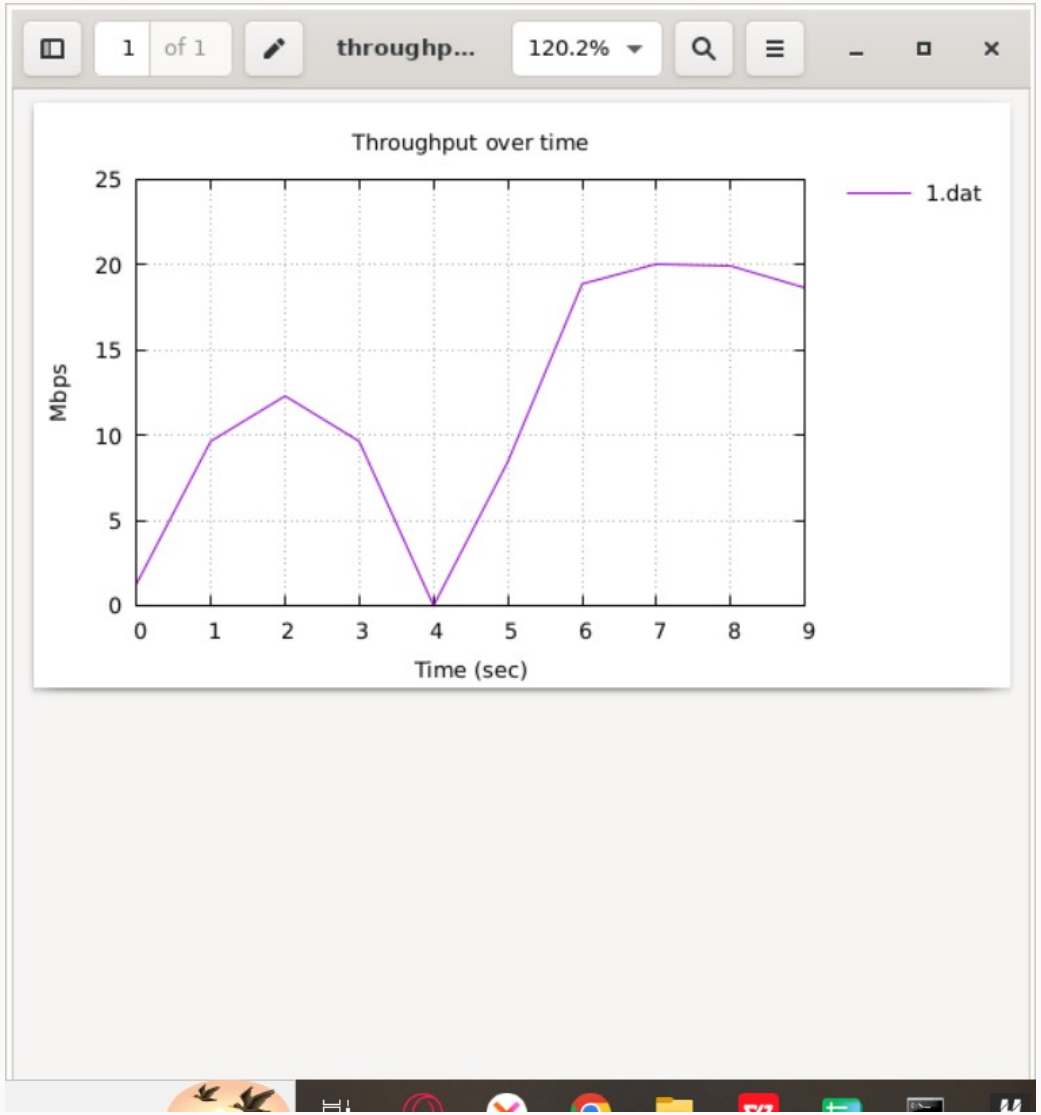


Рис. 24: График пропускной способности

Окно перегрузки TCP

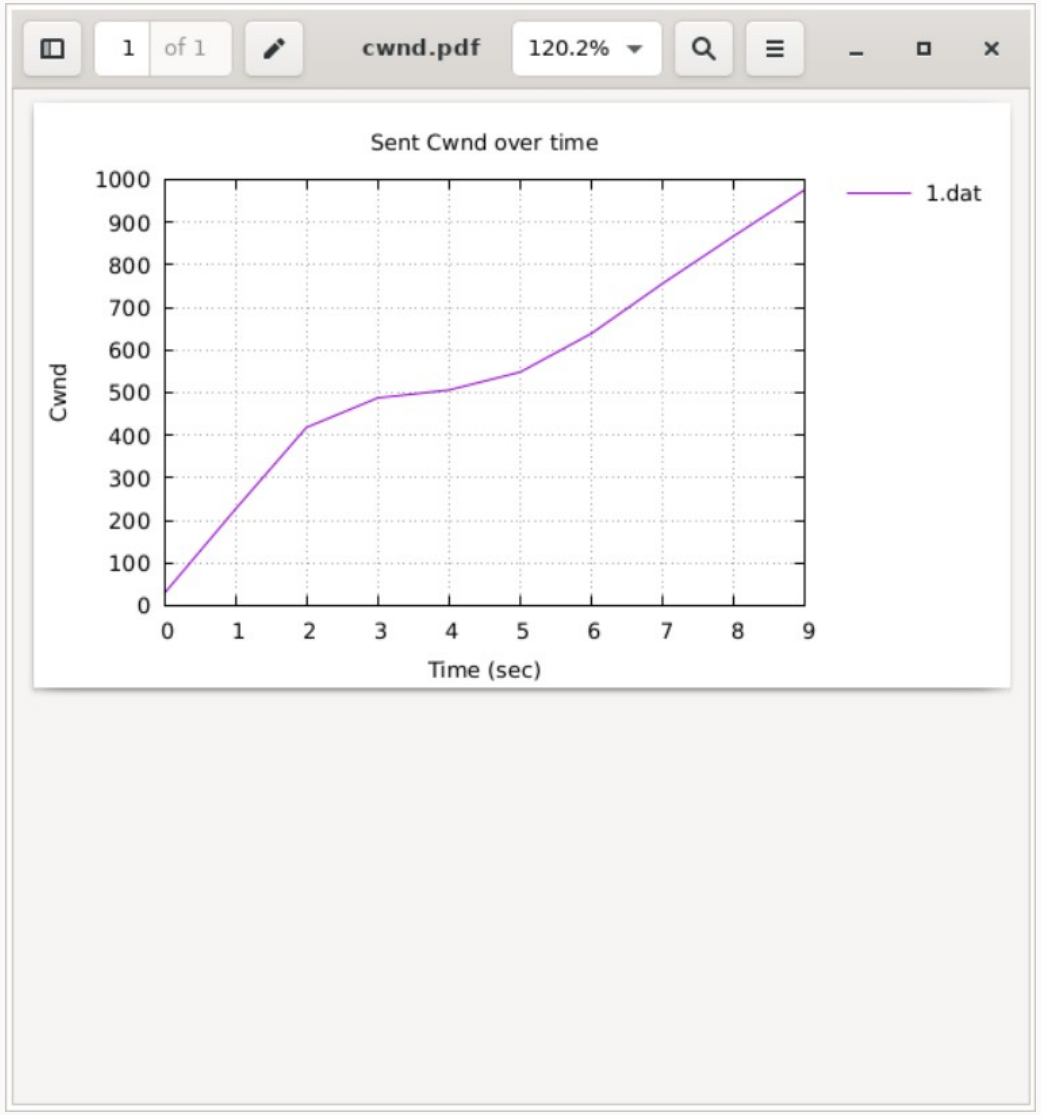


Рис. 25: График окна перегрузки TCP

Время подтверждения TCP (RTT)

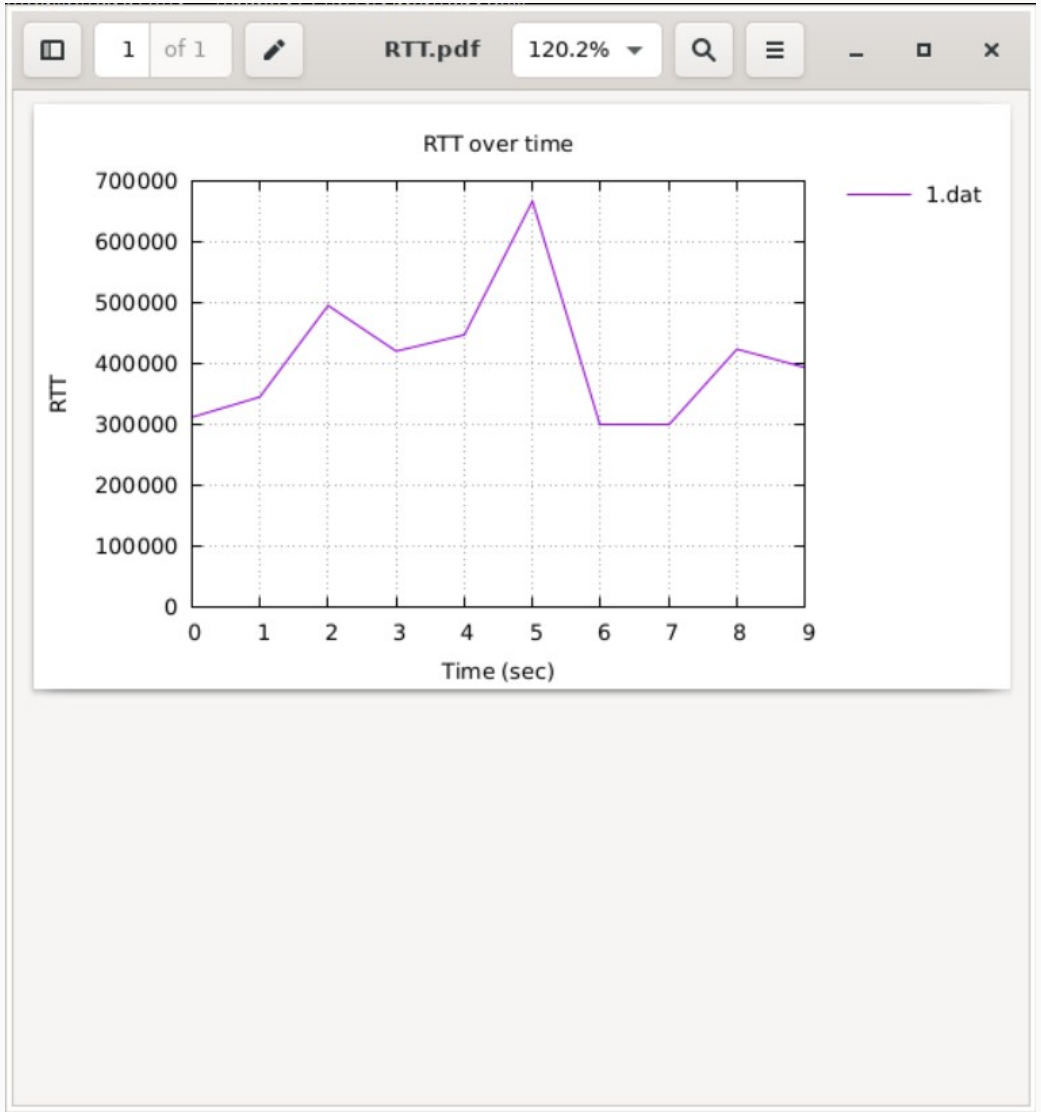


Рис. 26: График RTT

GNU Make и повторный запуск

Автоматизация с помощью Makefile

mininet@mininet-vm: ~/work/lab_iperf3/iperf3

GNU nano 4.8 Makefile Modified

```
.PHONY: all plot clean

all: iperf_result.json plot

iperf_result.json: lab_iperf3.py
	sudo python3 lab_iperf3.py

plot: iperf_result.json
	plot_iperf.sh iperf_result.json

clean:
	rm -f iperf_result.json *.csv
	rm -rf results
```

^G Get Help ^O Write Out ^W Where Is ^K Cut Text ^J Justify ^C Cur Pos
^X Exit ^R Read File ^\ Replace ^U Paste Text ^T To Spell ^_ Go To Line

Рис. 27: Правила автоматизации в Makefile

Повторный запуск эксперимента

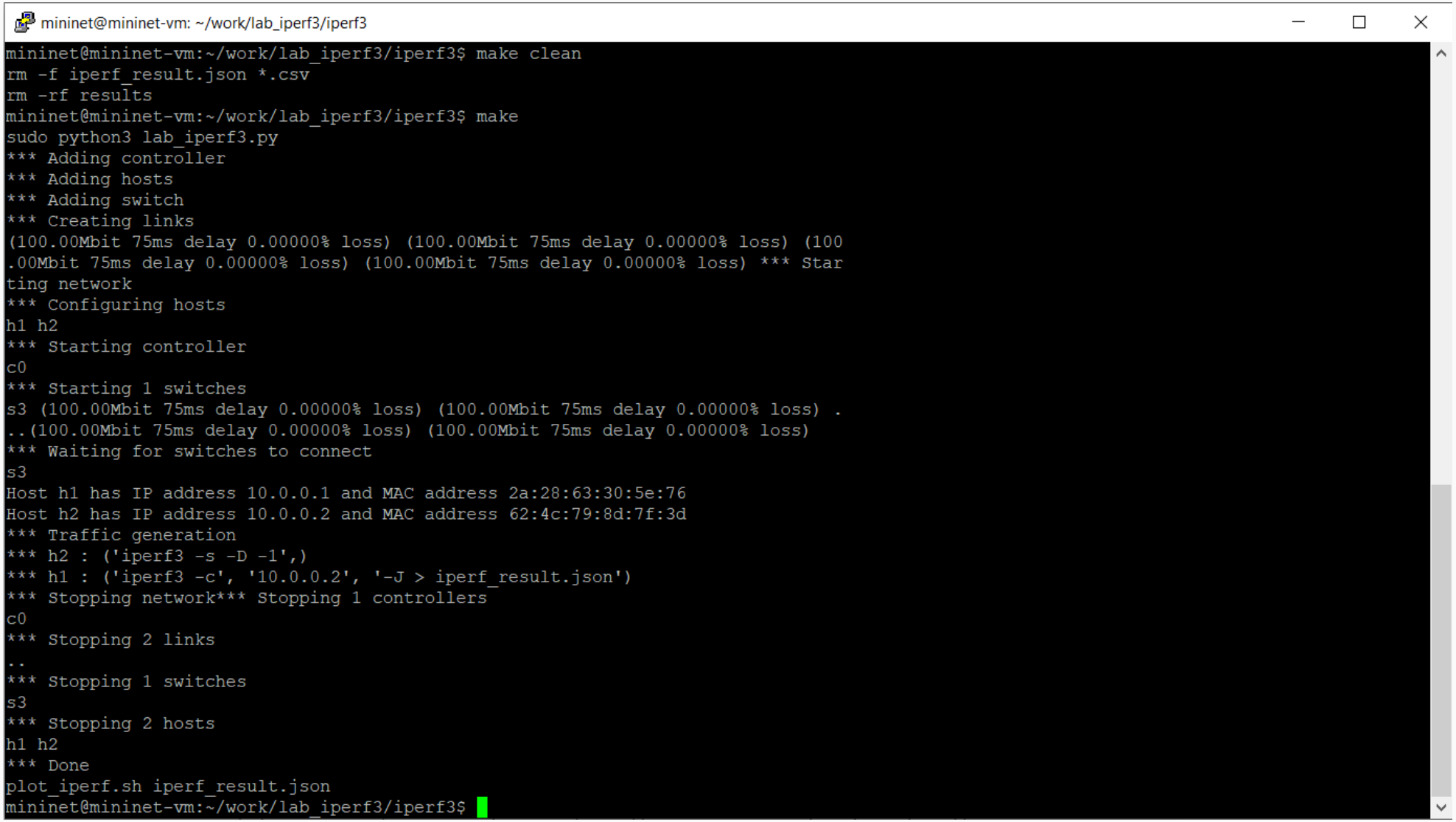


Рис. 28: Повторный запуск после очистки результатов

Проверка воспроизводимости результатов

```
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ ls -lh iperf_result.json results/
-rw-r--r-- 1 root    root    7.6K Oct 10 08:15 iperf_result.json

results/:
total 88K
-rw-rw-r-- 1 mininet mininet  527 Oct 10 08:15 1.dat
-rw-rw-r-- 1 mininet mininet 9.6K Oct 10 08:15 bytes.pdf
-rw-rw-r-- 1 mininet mininet 9.4K Oct 10 08:15 cwnd.pdf
-rw-rw-r-- 1 mininet mininet 8.9K Oct 10 08:15 MTU.pdf
-rw-rw-r-- 1 mininet mininet 8.8K Oct 10 08:15 retransmits.pdf
-rw-rw-r-- 1 mininet mininet 9.1K Oct 10 08:15 RTT.pdf
-rw-rw-r-- 1 mininet mininet 9.1K Oct 10 08:15 RTT_Var.pdf
-rw-rw-r-- 1 mininet mininet 9.5K Oct 10 08:15 throughput.pdf
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ python3 -m json.tool iperf_result.json > /dev/null && echo "JSON VALID"
JSON VALID
mininet@mininet-vm:~/work/lab_iperf3/iperf3$ python3 -c "import json; d=json.load(open('iperf_result.json')); print('Error:',d.get('erro
r','none')); print('Throughput Mbps:',round(d['end']['sum_received']['bits_per_second']/1e6,2))"
Error: none
Throughput Mbps: 35.62
mininet@mininet-vm:~/work/lab_iperf3/iperf3$
```

Рис. 29: Проверка итоговых файлов и результата

Выводы

Сопоставление результатов

ICMP

Обычная сеть

0% потерь · RTT 0,176 мс

Ограниченная сеть

15% потерь · RTT 187,525 мс

TCP / iPerf3

Первый запуск

9,97 Мбит/с

Повторный запуск

35,62 Мбит/с

Процедура запуска воспроизводима. Численные результаты различаются.

Итог лабораторной работы

- Построена топология Mininet через Python API
- Проверены сетевые адреса и связность хостов
- Смоделированы ограничения CPU, задержек и потерь
- Выполнены TCP-измерения iPerf3 и построены графики
- Проверен повторный автоматический запуск через GNU Make